🎮 Feature Engineering dla Machine Learning

Jak zamienić surowe statystyki e-sportowe w cechy, które kocha model 🏆

Python w Zastosowaniach Big Data | Wykład 7

Prowadzący: Pavlo Melnyk

📋 Plan wykładu

1. ML workflow — gdzie w ogóle mieści się feature engineering?
2. Dlaczego dobre cechy biją "lepszy" model
3. Brakujące wartości — strategie uzupełniania
4. Encoding zmiennych kategorycznych — OneHot / Label / Ordinal
5. Skalowanie i normalizacja — StandardScaler vs MinMaxScaler
6. Tworzenie nowych cech — ratio, binning, cechy wielomianowe
7. Niezbalansowane klasy — krótki przegląd
8. `Pipeline` + `ColumnTransformer` — jak to poskładać w całość
9. Train/test split i wycieki danych (data leakage)
10. Pułapki, podsumowanie, co dalej

> 🎯 **Po tym wykładzie:** Zbudujesz kompletny `Pipeline` scikit-learn, który zamienia brudne dane o graczach e-sportowych w cechy gotowe pod model ML — bez wycieku danych!

💡 Dlaczego to ważne?

W e-sporcie (League of Legends, CS2) drużyny analityczne przewidują **prawdopodobieństwo wygranej** gracza jeszcze przed meczem, na podstawie statystyk z ostatnich sezonów 🎮📊.

Ale surowe dane wyglądają tak:

player_id	rank	kda	reaction_ms	hours_week
4021	"Diamond"	3.2	NaN	14
4022	"brąz"	NaN	210	3

> Model nie "widzi" tekstu `"Diamond"` ani nie lubi `NaN`. Musimy to **przetłumaczyć** na język, który rozumie — liczby, bez śmieci i bez fałszywych sygnałów.

> 🔑 W praktyce data science: **80% czasu to feature engineering, 20% to trenowanie modelu.** Dziś robimy te 80%!

🔄 Gdzie w ML workflow jest feature engineering?

```
Surowe dane → Eksploracja (EDA) → Czyszczenie → Feature Engineering
           → Train/Test Split → Trenowanie modelu → Ewaluacja
```

Feature engineering to **most** między brudnymi danymi a modelem. Bez niego nawet `RandomForestClassifier` z domyślnymi parametrami nic nie zdziała.

> 💡 Kaggle grandmasterzy powtarzają: "*Better features beat better algorithms.*" — lepsze cechy biją lepszy algorytm!

🧪 Dzisiejszy dataset: statystyki graczy e-sportowych

Generujemy proceduralnie ~5000 graczy z realistycznymi korelacjami:

Kolumna	Opis
`matches_played`	Liczba rozegranych meczów
`win_rate`	Odsetek wygranych (0.0–1.0)
`kda`	Kill/Death/Assist ratio
`avg_reaction_ms`	Średni czas reakcji (ms)
`rank`	Kategoria: Brąz → Srebro → Złoto → Platyna → Diament → Mistrz → Wyzwanie
`hours_per_week`	Godziny gry tygodniowo
`country`	Kraj gracza

> 🎯 Wyższy `win_rate` koreluje z wyższym `kda` i większą liczbą `hours_per_week` — plus odrobina szumu, żeby było realistycznie!

🕳️ Brakujące wartości — dlaczego się pojawiają?

W realnych danych o graczach: awarie trackera statystyk, gracze bez wystarczającej liczby meczów, błędy importu z API.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "kda": [3.2, np.nan, 1.8, 2.5, np.nan],
    "avg_reaction_ms": [180, 210, np.nan, 195, 230],
    "rank": ["Diament", None, "Brąz", "Złoto", "Srebro"],
})

print(df.isnull().sum())  # 👈 Ile braków w każdej kolumnie?
```

🕳️ Strategie uzupełniania — `SimpleImputer`

```python
from sklearn.impute import SimpleImputer
import numpy as np

# Cechy numeryczne – mediana jest odporna na outliers
num_imputer = SimpleImputer(strategy="median")

# Cechy kategoryczne – najczęstsza wartość (moda)
cat_imputer = SimpleImputer(strategy="most_frequent")

data = np.array([[3.2, 180.0], [np.nan, 210.0], [1.8, np.nan]])
filled = num_imputer.fit_transform(data)
print(filled)  # 👈 Braki zastąpione medianą kolumny
```

> ⚠️ Nigdy nie uzupełniaj `mean()` / `median()` policzonym na **całym** zbiorze przed train/test split — to już wyciek danych! Wracamy do tego pod koniec wykładu.

🤔 Think About It

Pytanie: Masz kolumnę `avg_reaction_ms` z 2% brakujących wartości oraz kolumnę `country` z 40% brakujących wartości. Czy stosujesz tę samą strategię dla obu?

> 💡 Przy 40% braków czasem lepiej dodać kategorię `"nieznany"` niż zgadywać modą — sam brak informacji może być sygnałem!

🔤 Encoding kategoryczny — trzy podejścia

Model rozumie tylko liczby. `rank` to tekst — musimy go zakodować.

Metoda	Kiedy używać	Przykład
OneHotEncoder	Kategorie **bez** naturalnego porządku	`country`
LabelEncoder	Rzadko dla X; głównie do kodowania `y`	target klasyfikacji
OrdinalEncoder	Kategorie **z** naturalnym porządkiem	`rank` (Brąz < Srebro < ... < Wyzwanie)

> 🔑 `rank` ma wbudowaną hierarchię — idealny kandydat na `OrdinalEncoder` z ręcznie zdefiniowaną kolejnością!

🔤 OrdinalEncoder w praktyce

```
from sklearn.preprocessing import OrdinalEncoder
import numpy as np

rank_order = [["Brąz", "Srebro", "Złoto", "Platyna",
               "Diament", "Mistrz", "Wyzwanie"]]

encoder = OrdinalEncoder(categories=rank_order)
ranks = np.array(["Złoto", "Brąz", "Wyzwanie"]).reshape(-1, 1)
encoded = encoder.fit_transform(ranks)
print(encoded)  # [[2.], [0.], [6.]]  👈 Zachowany porządek!
```

> ✅ Dzięki jawnemu `categories=` model widzi, że Wyzwanie(6) > Złoto(2) > Brąz(0) — dokładnie tak, jak w grze.

🔤 OneHotEncoder — dla `country`

```python
from sklearn.preprocessing import OneHotEncoder
import numpy as np

encoder = OneHotEncoder(sparse_output=False, handle_unknown="ignore")
countries = np.array(["PL", "DE", "PL", "KR"]).reshape(-1, 1)
encoded = encoder.fit_transform(countries)

print(encoder.categories_)  # ['DE', 'KR', 'PL']
print(encoded)
# [[0. 0. 1.]   👈 PL
#  [1. 0. 0.]   👈 DE
#  [0. 0. 1.]   👈 PL
#  [0. 1. 0.]]  👈 KR
```

> ⚠️ `handle_unknown="ignore"` chroni przed crashem, gdy w zbiorze testowym pojawi się kraj, którego nie było w treningowym!

📏 Skalowanie — dlaczego różne skale to problem?

Cecha	Typowy zakres
`win_rate`	0.0 – 1.0
`avg_reaction_ms`	120 – 400
`matches_played`	10 – 3000

Bez skalowania `matches_played` "zdominuje" odległości w algorytmach opartych na dystansie (KNN, SVM, regresja logistyczna z regularyzacją).

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

scaler = StandardScaler()   # mean=0, std=1 — najczęstszy wybór
minmax = MinMaxScaler()     # skaluje do [0, 1] — dobre dla sieci neuronowych
```

📏 StandardScaler vs MinMaxScaler

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, MinMaxScaler

df = pd.DataFrame({
    "avg_reaction_ms": [150, 180, 400, 160, 220],
})

std_scaled = StandardScaler().fit_transform(df)
minmax_scaled = MinMaxScaler().fit_transform(df)

print(std_scaled.round(2).ravel())    # [-0.66 -0.35  2.36 -0.56  0.14... ]
print(minmax_scaled.round(2).ravel()) # [0.   0.12 1.   0.04 0.28]
```

> 🔑 `StandardScaler` — gdy dane ~normalne, drzewa nie wymagają skalowania w ogóle. `MinMaxScaler` — gdy chcesz stały zakres [0,1], np. sieci neuronowe.

🛠️ Tworzenie nowych cech — ratio

Surowe kolumny rzadko są najlepszymi cechami. Kombinujmy je!

```
import numpy as np

df["kda_per_hour"] = df["kda"] / df["hours_per_week"].replace(0, np.nan)
df["win_rate_x_kda"] = df["win_rate"] * df["kda"]  # 👈 interakcja cech

# "Efektywność" gracza — nowa cecha złożona
df["efficiency_score"] = (
    df["win_rate"] * 0.5 + (df["kda"] / df["kda"].max()) * 0.5
)
```

> 💡 Ratio i interakcje często ujawniają wzorce niewidoczne w pojedynczych kolumnach — jak "efektywność" łącząca win rate i KDA.

🛠️ Binning — zamiana liczb na kategorie

Czasem model lepiej radzi sobie z **przedziałami** niż z ciągłą liczbą.

```
import pandas as pd

df["hours_category"] = pd.cut(
    df["hours_per_week"],
    bins=[0, 5, 15, 30, 100],
    labels=["casual", "regularny", "hardcore", "no-life"],  # 👈 żartobliwie, ale czytelnie
)
print(df["hours_category"].value_counts())
```

> ⚠️ Binning traci informację (400h/tydzień i 31h/tydzień trafiają do tej samej kategorii "no-life") — używaj świadomie, gdy zależność jest nieliniowa.

🛠️ Cechy wielomianowe (Polynomial Features)

```
from sklearn.preprocessing import PolynomialFeatures
import numpy as np

X = np.array([[3.2, 180], [1.8, 220]])  # kda, reaction_ms
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)

print(poly.get_feature_names_out())
# ['kda' 'reaction_ms' 'kda^2' 'kda reaction_ms' 'reaction_ms^2']
```

> 🔑 Pomaga modelom liniowym łapać nieliniowe zależności — ale uważaj na **eksplozję liczby cech** przy wielu kolumnach i wysokim `degree`!

⚖️ Niezbalansowane klasy — krótki przegląd

Jeśli przewidujemy np. `"Wyzwanie" vs "reszta"`, klasa "Wyzwanie" może stanowić <2% danych.

Strategia	Opis
`class_weight="balanced"`	Model automatycznie waży klasy mniejszościowe
Oversampling (`SMOTE`)	Generuje syntetyczne przykłady klasy mniejszościowej
Undersampling	Zmniejsza liczbę przykładów klasy większościowej
Zmiana metryki	Zamiast accuracy: F1, precision-recall AUC

> 💡 Accuracy 98% brzmi świetnie, dopóki nie odkryjesz, że model po prostu zawsze przewiduje klasę większościową! 😅

🧩 `ColumnTransformer` — różne transformacje dla różnych kolumn

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

numeric_features = ["matches_played", "win_rate", "kda", "avg_reaction_ms"]
categorical_features = ["country"]
ordinal_features = ["rank"]

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])
```

🧩 `ColumnTransformer` — składamy w całość

```
preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_transformer, numeric_features),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features),
    ("ord", OrdinalEncoder(categories=[rank_order]), ordinal_features),
])

full_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(random_state=42)),
])

full_pipeline.fit(X_train, y_train)  # 👈 Jedna linia – cały pipeline!
```

> ✅ `Pipeline` gwarantuje, że **te same** transformacje (fit na train) zostaną zastosowane do danych testowych, bez ręcznego powtarzania kodu.

🎯 Train/Test Split — kolejność ma znaczenie!

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y  # 👈 zachowuje proporcje klas
)

# ❌ ŹLE: fit scalera na całym X przed splitem
# scaler.fit(X)  # Model "widzi" statystyki ze zbioru testowego!

# ✅ DOBRZE: fit tylko na X_train, potem transform na obu
full_pipeline.fit(X_train, y_train)
```

🚨 Data Leakage — najgroźniejsza pułapka

Wyciek danych (data leakage) — informacja ze zbioru testowego (lub z przyszłości) "przecieka" do procesu trenowania, sztucznie zawyżając wyniki.

Błąd	Dlaczego to wyciek
`scaler.fit()` na całym zbiorze przed splitem	Model "zna" rozkład danych testowych
Imputacja medianą liczoną na całości	Test wpływa na wartości treningowe
Feature stworzony z użyciem `y` (targetu)	Model oszukuje, bo target przecieka do X

> 🔑 **Złota zasada:** wszystko, co "uczy się" z danych (`fit`), robimy **wyłącznie** na `X_train` — nigdy na całym zbiorze!

⚠️ Częste pułapki Feature Engineering

Pułapka	Skutek	Rozwiązanie
`fit_transform` na całym zbiorze	Data leakage	`fit` tylko na train, `transform` na test
LabelEncoder na kolumnie nominalnej	Fałszywy porządek	OneHotEncoder
Ignorowanie `handle_unknown`	Crash na nowej kategorii	`handle_unknown="ignore"`
Zbyt duży `degree` w PolynomialFeatures	Eksplozja wymiarów	Ograniczaj `degree` i liczbę kolumn wejściowych
Usuwanie outlierów bez kontekstu	Utrata sygnału (np. pro-gracza)	Analizuj kontekst biznesowy

Podsumowanie

1. ✅ Feature engineering to często ważniejszy krok niż wybór modelu
2. ✅ Brakujące wartości: `SimpleImputer` (median/mode), z rozwagą
3. ✅ Encoding: `OneHotEncoder` (nominalne) vs `OrdinalEncoder` (uporządkowane)
4. ✅ Skalowanie: `StandardScaler` / `MinMaxScaler` — zawsze po split!
5. ✅ `Pipeline` + `ColumnTransformer` = powtarzalność i brak wycieku danych

🔮 Co dalej?

Dzisiejsze laboratorium: Zbudujesz pełny `Pipeline` feature engineering na zbiorze ~5000 graczy e-sportowych 🎮.

Wykład 8 (24.11.2026): Modele ML — klasyfikacja, regresja i ewaluacja. Wykorzystamy dziś przygotowane cechy do trenowania i oceny prawdziwych modeli!

❓ Pytania?

Dziękuję za uwagę! 🎉

Prowadzący: Pavlo Melnyk (melnyk@pjwstk.edu.pl)